

Better, Faster, Stronger: Programmatic Skill Learning Best Reduces Agent Cost

Zixi Huang^{*†} Xiheng Wang^{*} Andrew Wang^{*†}

William Jurayj Bernal Jiménez Gutiérrez Daniel Khashabi[†] Nicholas Andrews[†]

Johns Hopkins University
 {zhuang60, awang116, danielk, noa}@jhu.edu
^{*}Equal contribution [†]Corresponding authors

Abstract

Recently, the practice of augmenting LLM agent capability with *skills* has gained prevalence. We explore the cost effective adaptation of agents to novel domains by means of learning skills. Existing works focus on performance gain over cost effectiveness. As a result, little is known about what skill learning strategies save cost. We argue that among all the different skill learning methods, those that view skills as programs can achieve the best cost reduction. By executing sequences of actions deterministically, a program-augmented agent can reliably and cheaply achieve goals that would otherwise require trial and error and risk degenerate behavior over long horizons. An agent can learn at inference time by incrementally discovering these programs and equipping them for future tasks. We hypothesize that past trajectories contain enough signal to guide skill learning, even without replay or validation, provided the agent can learn to analyze them. To test our claims, we propose SpeedRunner, a coding agent that analyzes trajectories and refactors skills for better performance on future tasks. Across three different embodied environments, we show that SpeedRunner consistently achieves the frontier in learning and cost reduction while remaining robust against distribution shifts and environmental randomness.

1 Introduction

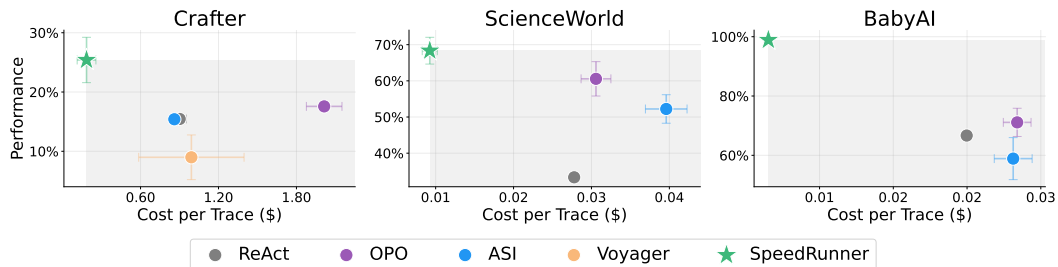


Figure 1: Performance vs average cost per trace for different approaches with GPT 5.4-mini on 3 benchmarks using learned programmatic skills. **SpeedRunner consistently dominates baselines in cost and performance, highlighting the potential advantages of programmatic skills.**

As LLM agents become more prevalent, they are increasingly deployed in complex real world environments. For instance, such agents now run a real world coffee shop in Stockholm.¹ Such deployments rarely present a fixed task distribution: new menu items and unfamiliar requests accumulate over time, so a static policy degrades unless the agent can adapt on the job. In these settings, we expect agents to learn *online* from their environment without catastrophically forgetting their pre-existing knowledge. We view *skill learning* as a promising paradigm. Here an agent reflects on its trajectories and updates its policy with environment-specific capabilities called *skills* (§2). The manner in which the policy is updated, the representation of skills, and the degree of online vs offline learning, vary between methods.

The predominant representation of skills are natural language descriptions (Zhong et al., 2026; Mi et al., 2026; Ni et al., 2026; Wang et al., 2026b; Yang et al., 2026), following the release of Anthropic’s agent skills . However, this shift has overlooked a key consideration for the average user: **cost**. Existing works focus on performance gains, with cost savings as an afterthought. Thus the effect of different skill representation and learning strategies on cost remains unknown. **We argue that the modality of code can be a more cost effective way to represent skills, compared to natural language.** Rather than reasoning about the same routine over and over again, an agent learns to offload that reasoning to a cheaper Turing machine. Hence, the more reasoning successfully translated into code, the more cost saved.

However, the key question is how well agents can learn programmatic skills from experience, and then successfully use them to accomplish tasks. Existing approaches fall short in multiple ways. First, the prevailing method of learning from experience has been to prompt an LLM with past trajectories. This method is critically bottlenecked by trajectory length; the number that fit in context decreases as they grow longer, until none fit at all. Second, unrealistic assumptions are made to facilitate learning, the most glaring of which assumes the ability to manipulate the environment to the extent of undoing and rewinding states. This often manifests as a set of held-out episodes used to gather heuristics for hillclimbing that are replayed repeatedly. Together, these shortcomings prevent the practical deployment of programmatic skill learning, especially in multi-turn agentic settings.

We hypothesize that the agent trajectories themselves are signal rich, and that simply analyzing them properly resolves these problems. **At a high level, we frame the process of extracting signal as an agentic coding task.** Given a history of trajectories, an agent with access to a code execution environment can programmatically identify recurring behavior patterns, skill usage and success metrics, performance regression, among others. Unlike natural language feedback (Agrawal et al., 2026; Pryzant et al., 2023) or handcrafted tools (Stengel-Eskin et al., 2024), a coding agent can avoid needle-in-a-haystack retrieval over long trace histories, support more structured cross-trajectory analysis, and mitigate the context-window bottleneck that arises in long-horizon environments with 200-step solutions (Hafner, 2022).

To test our hypothesis, we propose SpeedRunner, a simple yet effective way to induce programmatic skills using coding agents to analyze observed trajectories. Our goal is to optimize a stochastic policy parameterized by a library of skills, which, in practice, is an LLM agent equipped with a set of callable functions. The search for optimal skills cycles between two phases. The stochastic policy receives tasks and generates trajectories. Then, based on the trajectories, SpeedRunner updates the skills.

We evaluate SpeedRunner on ScienceWorld (Wang et al., 2022), Crafter (Hafner, 2022), and BabyAI (Chevalier-Boisvert et al., 2019)—a diverse set of embodied simulations. Each domain features vastly different dynamics, physical constraints, and action spaces. Nevertheless, we find consistent trends across all domains. (1) Programmatic skills can be learned online, without replay buffers or continual validation. (2) Their primary advantage is cost reduction without compromising performance. (3) Programmatic trajectory analysis enables skill induction for long-horizon agents. We also find that programmatic skill learning can be robust to environmental randomness and to distribution shifts.

2 Preliminaries

Problem formulation. We assume a partially observed Markov decision process (POMDP) and a set of valid initial states, where task *instructions* if available are part of the initial state. The POMDP we

¹<https://andonlabs.com/blog/ai-cafe-stockholm>

Algorithm 1 SpeedRunner: Wake–Sleep Skill Library Learning

Require: initial library L_0 , actor π_L , inducer $g : L \times H \rightarrow L$, environment E , task distribution $\mathcal{T}$, total episodes N , minibatch size k

```
1:  $H \leftarrow \emptyset$ 
2:  $L \leftarrow L_0$ 
3: for  $n = 1, \dots, N$  by  $k$  do
4:    $\triangleright$  wake phase
5:   sample tasks  $\{x_1, \dots, x_k\} \sim \mathcal{T}$ 
6:   for all  $x_i \in \{x_1, \dots, x_k\}$  do
7:      $\tau_i \sim \pi_L(x_i)$   $\triangleright$  rollout in  $E$  starting from task  $x_i$ 
8:   end for
9:    $B \leftarrow \{\tau_1, \dots, \tau_k\}$ 
10:   $H \leftarrow H \cup \{(B, L)\}$   $\triangleright$  tag trajectories with the generating library
11:   $\triangleright$  sleep phase
12:   $L \leftarrow g(L, H)$   $\triangleright$  inducer inspects trajectories and past libraries in  $H$ ; diagnoses failures/regressions; edits skills; sets public/private visibility
13: end for
14: return  $L$ 
```

trajectories in context. Because the raw trajectories are still saved, this avoids the signal loss introduced by summary-based compression.

Harness. The inducer acts as a programmer examining history and deciding what logic to implement. It must understand the environment, diagnose failures, assign credit to skills, and check for regressions. To support these analyses, we store both trajectories and past library versions. Each trajectory is indexed by the L version that generated it (line 10 of Algorithm 1), allowing the inducer g to analyze how functions evolve or regress over time. We also augment trajectories with call stacks for invoked skills: when a skill expands into a sequence of primitive actions, we record those intermediate actions and their outputs. These call stacks help diagnose silent skill failures.

Finally, we control library size. If the library grows without bound, the actor considers too many functions in context and performance degrades (Wang et al., 2025a). We therefore introduce public and private access modifiers. The inducer is instructed to make helper functions private and main functions public; private functions remain callable by other skills but are hidden from the actor.

4 Experimental Setup

4.1 Baselines

We compare four methods that differ in how they accumulate experience—none, prose, or code—while standardizing the primitive actions available to both the actor and inducer across all baselines.

ReAct This baseline serves as a control with no learning between episodes. The actor uses only the benchmark’s atomic action interface and whatever context is available within the current episode (Yao et al., 2023); it does not update any persistent strategy notes or reusable skills from past experience.

Online Prompt Optimization (OPO). We implement an online version of prompt optimization methods such as Agrawal et al. (2026); Yuksekogonul et al. (2025). After each training batch, a sleep-phase LLM reviews a concatenation of recent trajectories and updates the actor prompt.

ASI. Where supported, we also compare against Agent Skill Induction (ASI) (Wang et al., 2025c), a recent code-based baseline that induces skills from successful trajectories. Unlike SpeedRunner, ASI is append-only and verifies newly induced skills through replay-style checking on the inducing task instance. To satisfy our no-replay constraint, we remove this verification step. Appendix E details this adaptation, and Appendix G.3 reports an ablation of its effect.

Voyager. For Crafter, we additionally compare against Voyager (Wang et al., 2023), a code-based lifelong-learning agent that learns through an automatic curriculum and an append-only executable skill library. We evaluate Voyager only on Crafter because our online protocol removes the replay-and-retry behavior available in Voyager’s original setting: once a rollout ends, the agent cannot reset

to the same state, preserve inventory or other environment progress, and retry the same attempted skill. Under this constraint, Voyager’s curriculum is well defined only when the agent can continue pursuing a relatively stable objective across rollouts.

Crafter satisfies this requirement. Although each rollout starts in a new world, the high-level objective remains fixed: make progress in the survival environment. Voyager can therefore continue proposing and refining intermediate exploration goals across rollouts without the target distribution changing after every episode. ScienceWorld and BabyAI do not have this property. Their goals are externally specified and vary from episode to episode, so Voyager’s curriculum would be optimizing against a moving target. Evaluating Voyager on these benchmarks would require additional modification on the benchmark itself, making the comparison less faithful to the original algorithm. We therefore report Voyager only on Crafter, where a small adaptation is sufficient to fit our online setting; details are provided in Appendix F.

4.2 Selected Benchmarks

We evaluate on three text-based embodied benchmarks—ScienceWorld, BabyAI, and Crafter. Each defines its own physics, action space, and procedural demands, so success across all three is stronger evidence of generality than success across benchmarks that share a single underlying medium (e.g., compute-use benchmarks where the primitives reduce to file and terminal operations regardless of the task). We select subtasks where ReAct is neither trivial nor saturated, allowing us to isolate gains from skill induction.

4.2.1 ScienceWorld

ScienceWorld (Wang et al., 2022) is a text-based interactive environment grounded in elementary school science curricula with 10 task categories and 25 typed atomic actions (full list in Appendix B). An episode succeeds when the simulator score reaches 1.0; partial scores are binarized to failure.

Since skill reuse happens within category, we evaluate one category at a time. We choose Electricity (task 3) and Classification (task 4) because they are (a) solvable but not saturated by the out-of-the-box agent and (b) have enough tasks to measure learning over time.

4.2.2 BabyAI

BabyAI (Chevalier-Boisvert et al., 2019) is a procedurally generated 2D grid-world for instruction following under partial observability. We use the BabyAI-Text wrapper (Carta et al., 2023) as integrated in BALROG (Paglieri et al., 2025), which surfaces text observations. The agent acts through 6 primitives; episodes are scored as binary success.

Of BALROG’s 5 subtasks, we evaluate on `pick_up_seq_go_to`, which requires sequentially picking up an object and navigating to a goal in the correct order. The ReAct baseline already saturates the other four, leaving no headroom to measure gains from skill induction. We learn from 200 training episodes (full details in Appendix C).

4.2.3 Crafter

Crafter (Hafner, 2022) is an open-ended survival benchmark inspired by Minecraft, where agents gather resources, craft tools, and survive hazards in procedurally generated worlds. We expose 18 primitive actions and score each episode by the fraction of 22 predefined achievements unlocked before death or the 2,000-action budget. Unlike the BALROG (Paglieri et al., 2025) text wrapper, we expose full episode history and a richer textual view of the surroundings (full details in Appendix D).

4.3 Evaluation Setup

All experiments use gpt-5.4-mini for both the actor and inducer. Each run consists of 200 online rollouts with a sleep cycle every 10 rollouts. We impose a 30-minute wall-clock limit on each rollout to prevent nonterminating or pathologically long executions, such as loops inside generated skills. When the limit is reached, the rollout is terminated and scored using the environment state at termination; its token usage and partial task progress are retained in the reported metrics. All settings are repeated across 3 random seeds; we report means and ± 1 standard deviation across seed-level

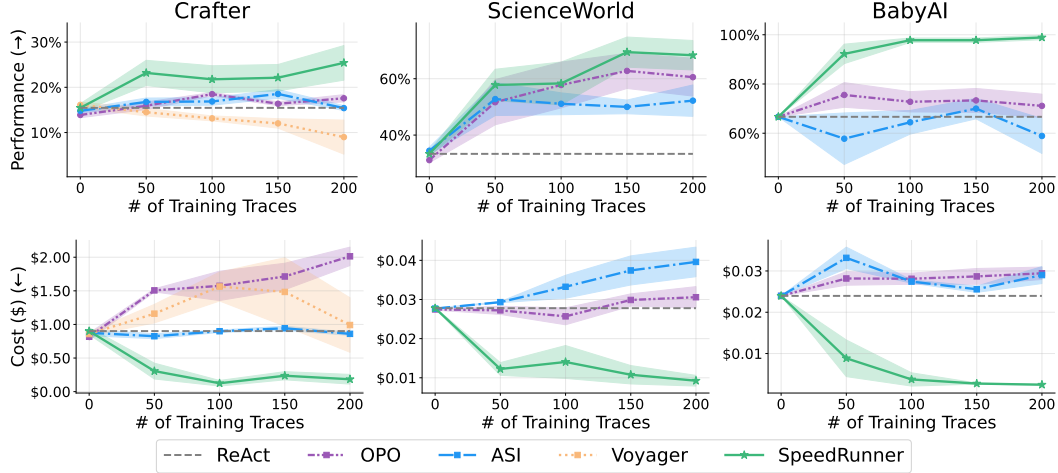


Figure 3: Performance and output-token cost across Crafter, ScienceWorld, and BabyAI. Our method and all baselines use GPT-5.4-mini as their backbone LLM. When the library overflows the model context limit, as happened for ASI, OPO and Voyager in Crafter, the sleep phase produces no changes and the process continues. **SpeedRunner significantly outperforms all baselines (\$4.1) across benchmarks in terms of performance and cost except for OPO in ScienceWorld on performance, per two-sided paired t-tests.**

results to reflect run-to-run variability. A fixed held-out test set of 30 episodes per benchmark is shared across all methods and checkpoints; evaluation is performed every 50 training rollouts. We evaluate along two axes: task progression (benchmark-specific success or achievement rate) and efficiency (output tokens per episode). Full implementation details are provided in Appendix A.

5 Results

Across all three benchmarks, SpeedRunner improves the performance–cost tradeoff of online skill induction. The clearest and most consistent effect is efficiency: SpeedRunner substantially reduces cost on every benchmark. Performance also improves, but less uniformly, suggesting that the value of executable skill induction depends on how much reusable behavior the environment exposes. All results below use GPT-5.4-mini, however, our cross-model experiments using Gemini-3-Flash and Qwen-3.5-27B in Appendix G.2 reveal that our method’s performance–cost tradeoff improvements hold in most settings.

Performance. SpeedRunner achieves the strongest final performance on all three benchmarks, with the largest margins in domains where learned routines transfer reliably across episodes. On BabyAI, SpeedRunner climbs from the $\sim 67\%$ ReAct baseline to near-perfect performance, a regime no other method reaches. On ScienceWorld, SpeedRunner ties OPO at the top and substantially outperforms ASI. Crafter is the harder case (every method ends below 30% mean progression) but SpeedRunner still leads. Appendix G.4 discusses why the magnitude of these gains varies with benchmark structure.

Efficiency. SpeedRunner is the only method whose cost *decreases* over training, and the reduction holds across all three benchmarks — most dramatically on BabyAI, where final usage falls to roughly an eighth of the ReAct baseline. This drop indicates that the library compresses repeated behavior into callable routines: the actor invokes high-level skills rather than re-deriving primitive sequences. ASI moves in the opposite direction on every benchmark, with cost growing as the library accumulates. OPO stays close to the ReAct baseline on ScienceWorld and BabyAI but grows substantially on Crafter, suggesting that prose memory cannot compress recurring patterns into reusable structure when the environment is high-entropy.

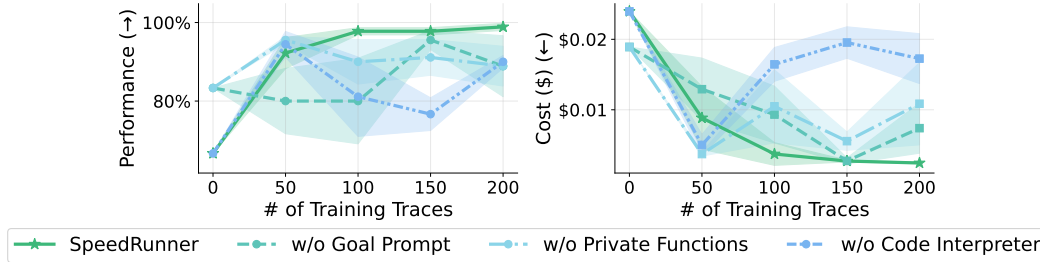


Figure 4: Ablating SpeedRunner components on the BabyAI environment. Individual components have only small effects on SpeedRunner’s performance, however, removing the code interpreter degrades efficiency considerably.

6 Discussion

6.1 Ablation Study

We ablate three components of SpeedRunner on BabyAI: the goal-oriented inducer prompt, the public/private function split, and the inducer’s code interpreter. The first ablation uses an ASI-style prompt that abstracts observed behavior rather than directly optimizing task solution; the second exposes all helper functions to the actor; the third replaces code-assisted trace analysis with in-context trajectory inspection. Due to experiment cost, full ablations are limited to BabyAI. Further discussion on the value of the code interpreter across environments can be found in Appendix G.1.

Figure 4 shows that all three ablations reduce performance, increase token usage, and introduce instability. Removing the code interpreter has the largest efficiency cost, suggesting that programmatic trace analysis is especially important for compression.

6.2 Code Complexity Analysis

6.2.1 Quantitative Analysis

We summarize the main quantitative findings here and defer the full structural statistics to Appendix H. For each final codebook, we construct a directed call graph $G = (V, E)$, where nodes are induced functions and edges denote function calls. We focus on two call-graph measures in Table 1: maximum depth and density. Maximum depth is the length of the longest directed path in G , measuring the deepest abstraction chain in the library. Density is $|E|/(|V|(|V| - 1))$, excluding self-edges, and measures how often induced functions call one another rather than only primitive actions.

Deeper and denser call graphs. Table 1 shows that SpeedRunner learns more structurally organized codebooks than the baselines. Across benchmark settings, SpeedRunner generally produces deeper and denser call graphs than ASI. This indicates that its learned skills are not merely independent wrappers around primitive actions, but are arranged into reusable hierarchical routines.

Reuse rather than accumulation. The contrast is especially clear on Crafter, where all methods are available. Voyager reaches maximum depth 5.7, but its density is only 0.0005, indicating little reuse among induced functions. In comparison, SpeedRunner reaches greater maximum depth (8.7) and much higher density (0.149). Thus, SpeedRunner does not improve by accumulating many scenario-specific functions; it factors repeated behavior into shared abstractions that later skills can call.

Overall, the quantitative signature of SpeedRunner is compositional organization: its functions call one another more often and form deeper abstraction hierarchies.

Method	Crafter		ScienceWorld		BabyAI	
	Max d.	Density	Max d.	Density	Max d.	Density
ASI	1.7±0.6	0.030±0.008	1.7±0.3	0.016±0.004	1.0±0.0	0.018±0.005
SpeedRunner w/o CI	6.3±3.2	0.138±0.078	5.2±1.2	0.073 ±0.013	3.3 ±1.5	0.178±0.019
SpeedRunner (ours)	8.7 ±2.1	0.149 ±0.042	5.8 ±0.8	0.067±0.018	2.3±0.6	0.206 ±0.042
Voyager	5.7±2.1	0.0005±0.0003	—	—	—	—

Table 1: Main call-graph statistics for final skill libraries, using GPT-5.4-mini with mean±sd over three seeds. The ScienceWorld column averages task 3 and task 4; reported means and standard deviations are averaged across the two tasks. We report maximum call-graph depth (Max d.) and call-graph density. **SpeedRunner learns deeper and denser call graphs.**

6.2.2 Qualitative Analysis

To understand where SpeedRunner’s gains come from, we now qualitatively compare our best-trial libraries against those of our baselines and ablations. Two patterns recur across all four environments; we summarize them here and defer per-environment evidence to Appendix I.

Code-enabled trace filtering. The inducer’s code interpreter lets it query specific slices of actor trajectories before committing a function. In BabyAI, candidate mission parsers were executed against historical mission strings to reveal that reversed-order phrasings (“go to X after you pick up Y ”) invert execution order—a failure mode that the ablation’s LLM-only inducer misses on 6.7% of episodes. In Crafter, the inducer noticed a high rate of not_found returns from travel_to_visible_resource, queried trace history to confirm the function was being called hundreds of times per batch while still failing, and introduced _bounded_exploration to sweep the local neighborhood for the desired resource before giving up. Without this loop, fixes accrue reactively one failure at a time.

Higher-level skills. The code interpreter enables abstractions that span high-level goals rather than a single primitive, a category of function essentially unique to SpeedRunner. ASI and Voyager produce primitives templated from individual traces (fixed motion sequences, per-resource collectors), with no high level goal logic. In contrast, SpeedRunner generates functions such as solve_sequential_mission (BabyAI), _find_visible_name_candidates (ScienceWorld), and _choose_progress_target (Crafter), each encoding a complex decision hierarchy in one place. The actor can often complete episodes with one or two high-level calls, explaining SpeedRunner’s 2–8× token reduction over ASI.

6.3 Effect of External Randomness (Crafter)

A central concern with code-based skill induction is that induced functions are *deterministic* by construction and therefore potentially brittle to environmental noise: each function commits to a fixed sequence of primitive actions, while real environments rarely repeat themselves exactly. We refer to this as the *external randomness gap*. Crafter lets us isolate this factor by varying the zombie spawn frequency — **0x** (no zombies), **1x** (standard), **2x** (doubled) — while holding all other mechanics fixed.

Figure 5 (bars) evaluates the robustness of all three methods to external randomness, measured against the ReAct baseline. Prose memory (OPO) remains useful under low randomness, but collapses once the environment becomes sufficiently stochastic. Append-only code (Voyager) shows a similar trend. In contrast, only SpeedRunner improves over the baseline in every condition.

The %Combat overlay (Figure 5, dotted lines) characterizes the learned codebooks. As zombie pressure increases, SpeedRunner does not simply expand its library; it reallocates representational budget based on the noisy signal, becoming deeper and more combat-aware by revising existing skills in place. This allows a single defensive abstraction to be amortized across the call graph. Voyager, by contrast, repeatedly reintroduces the same combat logic as duplicated branches inside newly generated, encounter-specific functions, without consolidating it into a general abstraction (Appendix J gives the detailed per-condition structural breakdown and examples).

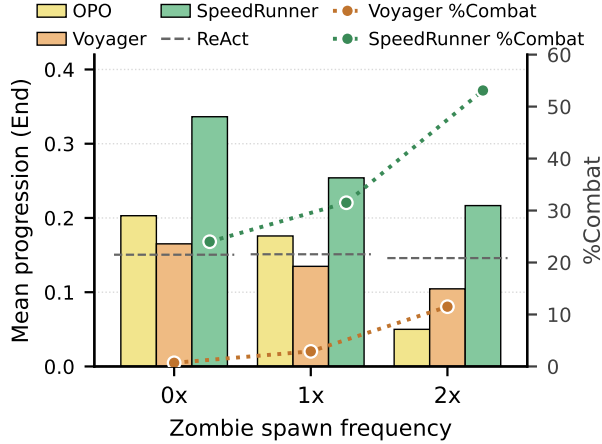


Figure 5: Crafter final performance vs *%Combat*, the fraction of induced functions whose body references a creature/combat keyword under different zombie settings (GPT-5.4-mini, 200 training rollouts).

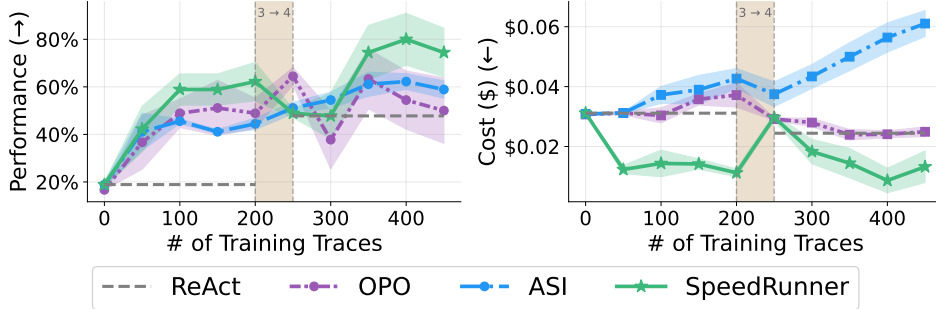


Figure 6: ScienceWorld distribution shift: performance (left) and cost (right) when switching from task 3 to task 4 midway through training. **SpeedRunner achieves the best performance and strongest token compression across both tasks.**

Together, these results suggest that SpeedRunner is better able to extract useful learning signal from stochastic traces. Rather than treating external randomness as irreducible noise, it identifies recurring failure modes and quickly incorporates the corresponding adaptations into reusable abstractions.

6.4 Adaptation under Distribution Shift (ScienceWorld)

Agents deployed in long-running environments must continually adapt to changing task distributions. A central challenge is to acquire new capabilities while retaining previously acquired ones. We study this setting by training on ScienceWorld task 3 (Electricity) for 200 rollouts before switching to task 4 (Classification), carrying the library over unchanged (Figure 6; GPT-5.4-mini, three seeds).

At the distribution shift, all methods’ performance and cost approach ReAct’s behavior because the two tasks require largely disjoint procedural knowledge. Throughout learning, SpeedRunner achieves the best performance and strongest token compression on both tasks. By the end of task 4, it refines a compact library of approximately 40 functions, compared to ASI which accumulates approximately 160 functions and plateaus at lower performance.

To evaluate knowledge retention, we re-evaluate task 3 after task 4 training. Despite actively revising its library, SpeedRunner loses only 3.3 percentage points on task 3 while reducing inference cost by 27.1%, making it the only method that becomes more efficient on the original task after adapting to a new distribution. In contrast, ASI improves task 3 performance by 4.4 percentage points but increases inference cost by 40.6% because of its append-only nature: it preserves all learned knowledge, at

the cost of unbounded library growth. OPO loses 5.6 percentage points on task 3 while increasing inference cost by 6.8%, performing worse than SpeedRunner in both retention and efficiency.

7 Related Work

Our work builds on the broader literature on continual learning (Kirkpatrick et al., 2017; Li & Hoiem, 2017; Wang et al., 2024; Shi et al., 2024) and self-evolving agents (Fang et al., 2025; ang Gao et al., 2026), which aim to build agents that improve continuously through interaction with their environment. Existing approaches broadly follow two lines: adapting the model’s weights (Qu et al., 2025; Song et al., 2024; Wang et al., 2025b) or augmenting a frozen model with externally stored knowledge (Shinn et al., 2023; Wang et al., 2023; Zhao et al., 2024). We study the latter, specifically *skill learning*, which distills experience into reusable skills loaded into context to help complete a task. Prior skill-learning methods primarily aim to improve task performance and adaptation, while treating inference cost as a secondary outcome. We instead study skill learning through the lens of inference efficiency, where the central design choice is how skills are represented.

Most methods represent skills in natural language. A line of work distills trajectories into reusable natural-language skill documents temporarily loaded into context when relevant (Zhao et al., 2024; Wang et al., 2025d; Ma et al., 2026; Wang et al., 2026a; Liu et al., 2026; Ni et al., 2026; Alzubi et al., 2026). Some methods adopt skill folders which bundle textual instructions and helper scripts (Liu et al., 2026; Alzubi et al., 2026), but the primary optimization target remains the natural-language artifact. While natural-language skills are flexible and generalize well, the agent must reread and re-follow the instructions on every reuse, so recurring procedures keep incurring inference cost.

The second line of work represents reusable skills as executable code. Foundational works used Bayesian program induction to synthesize formal representations through bottom-up composition of primitives (Ellis et al., 2018, 2021). Recent work extends this paradigm to LLM agents: Voyager grows a skill library through open-ended Minecraft exploration (Wang et al., 2023), while a wave of systems induce and refine skills from agent trajectories (Stengel-Eskin et al., 2024; Wang et al., 2025c; Zheng et al., 2025; Qiu et al., 2026). Programmatic skills replace repeated reasoning with delegated computation and thus provide a more efficient representation for recurring procedures. However, previous work primarily studies programmatic skills to improve agent capability. Many methods further rely on experience replay for validating candidate skills to guard against regression, yet these assumptions are difficult to satisfy in deployed environments where interactions are irreversible.

Our work similarly represents skills as executable code but differs in both objective and methodology. Rather than treating programmatic skills primarily as a means of improving capability, we highlight their capacity to minimize inference cost by maximizing reusable computation. We further show that such skills can be induced entirely online without replay by keeping the accumulated trajectories available as a growing dataset for programmatic analysis, enabling continual refinement of the skill library while mitigating regression.

8 Conclusion

In this work, we demonstrate that representing skills as code can achieve the best cost effectiveness while retaining performance. We observe that this trend is remarkably consistent across distribution shifts, randomness, different domains, and even different models. We further demonstrate that this trend exists in a purely-online setting, without a held-out set of episodes to evaluate skill update quality for hillclimbing. Not all programmatic skill learning approaches reduce cost to the same extent. We are only able to do so by framing the process of extracting signal from past trajectories as an agentic coding task.

Limitations We were unable to benchmark flagship models due to cost limitations. For the same reason, we were unable to evaluate on more expensive benchmarks such as SWE-Bench (Jimenez et al., 2024) or Terminal-Bench (Merrill et al., 2026). Nevertheless, we believe that the diversity and controllability of our evaluations are sufficient to support our general conclusions. Skill learning at test time suffers from instability, as demonstrated by the large error bars across all methods we evaluated; stabilizing online skill induction remains an open problem. Lastly, we do not study meta-learning of the inducer itself—in principle, the inducer could learn to perform better analyses over time, but we leave this to future work.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning, 2026. URL <https://arxiv.org/abs/2507.19457>.
- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL <https://arxiv.org/abs/2603.02766>.
- Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, Qihan Ren, Cheng Qian, Zhenhailong Wang, Minda Hu, Huazheng Wang, Qingyun Wu, Heng Ji, and Mengdi Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence, 2026. URL <https://arxiv.org/abs/2507.21046>.
- Thomas Carta, Clément Romac, Thomas Wolf, Sylvain Lamprier, Olivier Sigaud, and Pierre-Yves Oudeyer. Grounding large language models in interactive environments with online reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23*. JMLR.org, 2023.
- Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: First steps towards grounded language learning with a human in the loop. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJeXCo0cYX>.
- Kevin Ellis, Lucas Morales, Mathias Sablé-Meyer, Armando Solar-Lezama, and Joshua B. Tenenbaum. Library learning for neurally-guided Bayesian program induction. In *Proceedings of the 32nd International Conference on Neural Information Processing Systems, NIPS’18*, pp. 7816–7826, Red Hook, NY, USA, December 2018. Curran Associates Inc. URL <https://dl.acm.org/doi/10.5555/3327757.3327878>.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation, PLDI 2021*, pp. 835–850, New York, NY, USA, June 2021. Association for Computing Machinery. ISBN 978-1-4503-8391-2. doi: 10.1145/3453483.3454080. URL <https://dl.acm.org/doi/10.1145/3453483.3454080>.
- Jinyuan Fang, Yanwen Peng, Xi Zhang, Yingxu Wang, Xinhao Yi, Guibin Zhang, Yi Xu, Bin Wu, Siwei Liu, Zihao Li, Zhaochun Ren, Nikos Aletras, Xi Wang, Han Zhou, and Zaiqiao Meng. A comprehensive survey of self-evolving ai agents: A new paradigm bridging foundation models and lifelong agentic systems, 2025. URL <https://arxiv.org/abs/2508.07407>.
- Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=1W0z96MFEoH>.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN 1091-6490. doi: 10.1073/pnas.1611835114. URL <http://dx.doi.org/10.1073/pnas.1611835114>.

- Zhizhong Li and Derek Hoiem. Learning without forgetting, 2017. URL <https://arxiv.org/abs/1606.09282>.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. Skillforge: Forging domain-specific, self-evolving agent skills in cloud technical support, 2026. URL <https://arxiv.org/abs/2604.08618>.
- Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. SkillClaw: Let Skills Evolve Collectively with Agentic Evolver, April 2026. URL <http://arxiv.org/abs/2604.08377>. arXiv:2604.08377 [cs].
- Mike A Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Kwesi Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Jenia Jitsev, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. In *The Fourteenth International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=a7Qa4CcHak>.
- Qirui Mi, Zhijian Ma, Mengyue Yang, Haoxuan Li, Yisen Wang, Haifeng Zhang, and Jun Wang. Skill-pro: Learning reusable skills from experience via non-parametric PPO for LLM agents. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=9kJQjx2B80>.
- Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjin Jiang. Trace2skill: Distill trajectory-local lessons into transferable agent skills, 2026. URL <https://arxiv.org/abs/2603.25158>.
- Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=fp6t3F669F>.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search, 2023. URL <https://arxiv.org/abs/2305.03495>.
- Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: From trajectories to reusable expertise for continual llm agent refinement, 2026. URL <https://arxiv.org/abs/2601.22758>.
- Yuxiao Qu, Anikait Singh, Yoonho Lee, Amrith Setlur, Ruslan Salakhutdinov, Chelsea Finn, and Aviral Kumar. Rlad: Training llms to discover abstractions for solving reasoning problems, 2025. URL <https://arxiv.org/abs/2510.02263>.
- Haizhou Shi, Zihao Xu, Hengyi Wang, Weiye Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. Continual learning of large language models: A comprehensive survey, 2024. URL <https://arxiv.org/abs/2404.16789>.
- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.

- Yifan Song, Da Yin, Xiang Yue, Jie Huang, Sujian Li, and Bill Yuchen Lin. Trial and error: Exploration-based trajectory optimization for llm agents, 2024. URL <https://arxiv.org/abs/2403.02502>.
- Elias Stengel-Eskin, Archiki Prasad, and Mohit Bansal. ReGAL: Refactoring Programs to Discover Generalizable Abstractions, June 2024. URL <http://arxiv.org/abs/2401.16467>. arXiv:2401.16467 [cs].
- Andrew Wang, Sophia Hager, Adi Asija, Daniel Khashabi, and Nicholas Andrews. Hell or high water: Evaluating agentic recovery from external failures. In *Second Conference on Language Modeling*, 2025a. URL <https://openreview.net/forum?id=Zk224WPT42>.
- Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. Skillx: Automatically constructing skill knowledge bases for agents, 2026a. URL <https://arxiv.org/abs/2604.04804>.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*, November 2023. ISSN 2835-8856. URL <https://openreview.net/forum?id=ehfRiF0R3a>.
- Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. Skill-sd: Skill-conditioned self-distillation for multi-turn llm agents, 2026b. URL <https://arxiv.org/abs/2604.10674>.
- Liyuan Wang, Xingxing Zhang, Hang Su, and Jun Zhu. A comprehensive survey of continual learning: Theory, method and application, 2024. URL <https://arxiv.org/abs/2302.00487>.
- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (eds.), *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pp. 11279–11298, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.775. URL <https://aclanthology.org/2022.emnlp-main.775/>.
- Zihan Wang, Kangrui Wang, Qineng Wang, Pingyue Zhang, Linjie Li, Zhengyuan Yang, Xing Jin, Kefan Yu, Minh Nhat Nguyen, Licheng Liu, Eli Gottlieb, Yiping Lu, Kyunghyun Cho, Jiajun Wu, Li Fei-Fei, Lijuan Wang, Yejin Choi, and Manling Li. Ragen: Understanding self-evolution in llm agents via multi-turn reinforcement learning, 2025b. URL <https://arxiv.org/abs/2504.20073>.
- Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks, 2025c. URL <https://arxiv.org/abs/2504.06821>.
- Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent Workflow Memory. June 2025d. URL <https://openreview.net/forum?id=NTAh2JEEE>.
- Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026. URL <https://arxiv.org/abs/2603.01145>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. Optimizing generative ai by backpropagating language model feedback. *Nature*, 639:609–616, 2025.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144>.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL <https://arxiv.org/abs/2504.07079>.

Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo F. R. Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks, 2026. URL <https://arxiv.org/abs/2604.20087>.

A Implementation Details

A.1 Models and Decoding

All results in the main paper use gpt-5.4-mini as the underlying LLM for every role. Decoding settings differ by role:

- *Actor agent*: reasoning_effort = low, parallel_tool_calls = false, request limit 100 per episode.
- *Inducer agent*: reasoning_effort = medium, parallel_tool_calls = false, request limit 100 per episode.

Libraries are reset to empty at the start of training for each task family; only the primitives carry over. During the sleep cycle, the inducer is allowed to add, edit, or remove library entries between sleep cycles.

A.2 Training and Evaluation Schedule

Each training run consists of 200 online rollouts. The inducer is invoked every 10 rollouts (one sleep cycle per 10 training episodes), and the resulting library is evaluated on the 30-episode test set every 50 training rollouts, giving five evaluation checkpoints per run (zero-shot at rollout 0, plus rollouts 50, 100, 150, 200). Each setting is repeated over 3 random seeds. At each checkpoint, we first compute each seed-level metric over the 30 test episodes, then report the mean and ± 1 , s.d. across the three seed-level values.

A.3 Cost Accounting

Per-episode inference cost is computed as

$$\text{cost} = p_{\text{in}} N_{\text{in}}^{\text{uncached}} + p_{\text{cache}} N_{\text{in}}^{\text{cached}} + p_{\text{out}} N_{\text{out}},$$

where $N_{\text{in}}^{\text{uncached}}$, $N_{\text{in}}^{\text{cached}}$, and N_{out} are the uncached input, cached input, and output tokens summed over all LLM calls in the episode. This includes actor calls during rollout and sleep-time inducer calls amortized over the rollouts in the corresponding inducer batch. We use the published API prices as of May 6, 2026. For gpt-5.4-mini, we set

$$p_{\text{in}} = \$0.75, \quad p_{\text{cache}} = \$0.075, \quad p_{\text{out}} = \$4.50$$

per million tokens.

For gemini-3-flash-preview, we set

$$p_{\text{in}} = \$0.35, \quad p_{\text{cache}} = \$0.0875, \quad p_{\text{out}} = \$1.05$$

per million tokens.

We use this dollar-cost accounting in Figure 3.

B ScienceWorld Details

B.1 Chosen Tasks

We evaluate on two ScienceWorld task families: Electricity (family 3) and Classification (family 4). Each family contains multiple task names (sub-tasks); episodes are addressed by

(*task_name*, *variation_idx*) pairs. We use ScienceWorld’s built-in train/dev/test variation partitions (`get_variations_train/dev/test`), which are disjoint by construction. For each family we sample **200** train variations and **30** test variations with a fixed seed (42); the same train/test sets are reused across all three training seeds, so only the rollout stochasticity differs across runs.

B.2 Action Interface

The agent acts through a fixed set of 25 primitive Python functions that wrap ScienceWorld’s text parser. These primitives cover navigation (`go`, `look_around`, `look_at`, `look_in`), object manipulation (`pick_up`, `put_down`, `move`, `pour`, `dunk`), state changes (`open`, `close`, `activate`, `deactivate`, `connect`, `disconnect`, `mix`), measurement and inspection (`focus_on`, `read`, `use`, `inventory`, `task_description`), and auxiliary actions (`eat`, `flush`, `wait`, `disambiguate`). Each primitive takes typed string arguments (e.g. `connect(obj_a, obj_b)`); this is the interface the induced library composes against. We do not expose ScienceWorld’s `get_valid_actions` list to the agent. In our setting, the agent’s effective action space includes both primitives and any library functions available under the current method; surfacing only the primitive-level valid-action set would be misleading, since it omits the higher-level abstractions the agent is expected to prefer.

B.3 Observations and Episode Termination

Observations are plain text: on each step the agent receives the room description and the parser’s acknowledgement of the previous action. The full observation history is kept in context (no sliding window). The initial prompt contains the task description returned by ScienceWorld’s `taskdescription()` and the opening room observation; no oracle hints or gold trajectories are provided.

An episode ends when (i) the actor agent calls the `submit_answer` tool to declare the task complete, or (ii) the actor agent exceeds its budget of **100** LLM calls per episode. We set this budget at 100 because additional tool calls no longer meaningfully improve the agent’s success rate, so further extending the budget would inflate cost without changing the headline results. In both cases, success is determined by checking ScienceWorld’s internal score: the episode is scored as a success iff the score has reached its maximum value of 1.0, and as a failure otherwise.

C BabyAI Details

C.1 Chosen Tasks

We evaluate on the `pick_up_seq_go_to` subtask of BabyAI-Text as exposed by BALROG. Episodes are addressed by environment seed. We sample **200** train seeds and **30** test seeds with a fixed top-level seed (42); train and test seed sets are disjoint by construction. The same train/test sets are reused across all three training seeds, so only the rollout stochasticity differs across runs.

C.2 Action Interface

The agent acts through a fixed set of 6 primitive Python functions that wrap BabyAI-Text’s command interface: navigation (`turn_left`, `turn_right`, `go_forward`), object manipulation (`pick_up`, `drop`), and state change (`toggle`). Each primitive takes no arguments and operates on the cell directly in front of the agent; this is the interface the induced library composes against. We do not expose BALROG’s valid-action list to the agent. In our setting, the agent’s effective action space includes both primitives and any library functions available under the current method; surfacing only the primitive-level valid-action set would be misleading, since it omits the higher-level abstractions the agent is expected to prefer.

C.3 Observations and Episode Termination

Observations are plain text descriptions of the agent’s egocentric view, as produced by the BabyAI-Text wrapper. The full observation history is kept in context (no sliding window). The initial prompt contains the synthetic natural-language instruction sampled by BabyAI’s Baby Language grammar and the opening observation.

An episode ends when (i) the simulator emits a terminal signal indicating the instruction has been satisfied, or (ii) the actor agent exceeds its budget of **30** LLM calls per episode. We set this budget at 30 because additional tool calls no longer meaningfully improve the agent’s success rate, so further extending the budget would inflate cost without changing the headline results. Like ScienceWorld, success is determined entirely by the environment’s binary task-completion signal: the episode is scored as a success if the instruction is fully satisfied within the budget, and as a failure otherwise.

D Crafter Details

D.1 Episode Sampling

Each Crafter episode is a freshly generated world keyed by a single integer seed. We use three independent runs, each with **200** training worlds and **30** test worlds, but with different top-level seeds to ensure diversity. All methods are run three times using the same training and test worlds, so comparisons are matched by world seed. Unless stated otherwise, reported means and uncertainty bands are computed over these three matched runs.

D.2 Action Interface

The agent acts through a fixed set of 18 primitive Python functions wrapping the Crafter engine: 17 are game-advancing primitives and one is a non-advancing read action. These primitives cover navigation (move_north, move_south, move_east, move_west), interaction (do – a multi-use action that collects material, drinks from a lake, or hits the creature in front; sleep; noop – advance one tick without acting), structure placement (place_stone, place_table, place_furnace, place_plant), and tool crafting (make_wood_pickaxe, make_stone_pickaxe, make_iron_pickaxe, make_wood_sword, make_stone_sword, make_iron_sword). All 17 game-advancing primitives are zero-argument and return None; each advances the Crafter world by exactly one tick. The get_current_observation() primitive returns the current textual observation without advancing the world, and is provided so that library functions can re-inspect the world between primitive calls. Crafter also has no submit_answer action; episodes terminate only through the environment itself (see below).

D.3 Observations

Observations are plain text rendered from the agent’s egocentric 9×9 local view, with three blocks shown each step: vital status, inventory contents, and visible objects (with the object directly in front of the agent reported separately). Each observation is wrapped between «OBSERVATION_BEGIN» and «OBSERVATION_END» delimiters and substituted once into the {obs} slot of the user prompt template at the start of the rollout. It will also be returned upon agent calling the get_current_observation() primitive.

A representative mid-episode observation looks as follows:

```
<<OBSERVATION_BEGIN>>
Your status :
- health: 7/9
- food: 5/9
- drink: 6/9
- energy: 8/9
Your inventory :
- wood: 3
- stone: 1
- wood_pickaxe: 1
You see :
- grass 1 step north
- tree 2 steps north and 1 step east
- stone 3 steps east
- zombie 4 steps north and 2 steps west
- water 5 steps south and 3 steps east
You face grass at your front.
<<OBSERVATION_END>>
```


When the agent’s inventory is empty, the inventory block collapses to the line “*You have nothing in your inventory.*”; when nothing other than the player is visible in the 9×9 window, the visible-objects block collapses to “*You see nothing away from you.*” Sleeping or dead states are surfaced by prefixing the status block with a single sentence (e.g., “*You are sleeping, and will not be able to take actions until energy is full.*”).

D.4 Episode Termination

An episode ends when (i) the player’s health reaches zero, or (ii) the agent exhausts its budget of **2,000** primitive actions. Each episode is scored as the fraction of the 22 achievements unlocked at termination. The actor agent’s per-episode LLM-call budget (`request_limit`) is also set to 2,000 to ensure that code baselines are not disadvantaged by being cut off early.. In practice, we rarely see any rollout exceeding this budget.

D.5 Achievements

The agent is told about the 22 Crafter achievements in the initial system prompt, in a fixed display order: Collect Wood, Place Table, Eat Cow, Collect Sapling, Collect Drink, Make Wood Pickaxe, Make Wood Sword, Place Plant, Defeat Zombie, Collect Stone, Place Stone, Eat Plant, Defeat Skeleton, Make Stone Pickaxe, Make Stone Sword, Wake Up, Place Furnace, Collect Coal, Collect Iron, Make Iron Pickaxe, Make Iron Sword, Collect Diamond.

D.6 Zombie-Frequency Variants

The external-randomness analysis in §6.3 sweeps a single environment knob, `zombie_frequency` (f), while holding all other settings fixed. The knob enters the upstream Crafter engine in two places. First, at world reset: when $f = 0$, every zombie generated by terrain initialization is explicitly removed before the first step. Second, at every tick during world balancing: zombies spawn with probability $\min(1.0, 0.3f)$ and despawn with probability $\min(1.0, 0.4f)$ per eligible chunk, and the per-chunk target population is scaled by f . With this scaling, $f = 1.0$ recovers the upstream Crafter defaults, $f = 0.0$ removes zombies entirely, and $f = 2.0$ doubles both spawn and despawn pressures (each clipped at 1.0) and doubles the target population. No other engine parameters are changed across variants.

For the external randomness experiments, we instantiate this knob at three levels labeled **0x** ($f = 0.0$), **1x** ($f = 1.0$, default), and **2x** ($f = 2.0$), and run each of the three comparison methods at each level: OPO, Voyager, and SpeedRunner. For a given method, the 1x configuration is the same as the one used in the main results; therefore we directly reused the results from the main experiments; the 0x and 2x configurations differ only in `zombie_frequency`.

E ASI Adaptation Details

We adapt ASI (Wang et al., 2025c) to our *purly*-online setting while preserving its programmatic skill induction mechanism. In the original algorithm, each newly induced skill undergoes a *skill verification* step, in which the agent attempts to re-solve the inducing task using the new skill and retains the skill only if verification succeeds. Because this step requires additional task interaction beyond the original online trajectory, we remove it to satisfy our no-replay constraint. All other components of ASI are left unchanged.

F Voyager Adaptation Details

Given its relevance to our work, we make three task-level adaptations to preserve Voyager’s (Wang et al., 2023) core ingredients while aligning its training protocol with our online constraints. First, the curriculum agent is informed about the Crafter’s 22 achievements as the final optimization target when proposing the next intermediate subgoals. Second, we remove Voyager’s respawn-style continuation after death: when the agent dies or the episode terminates, training restarts from a fresh Crafter world with no inventory carryover, matching the irreversible online setting used by all other methods. Third,

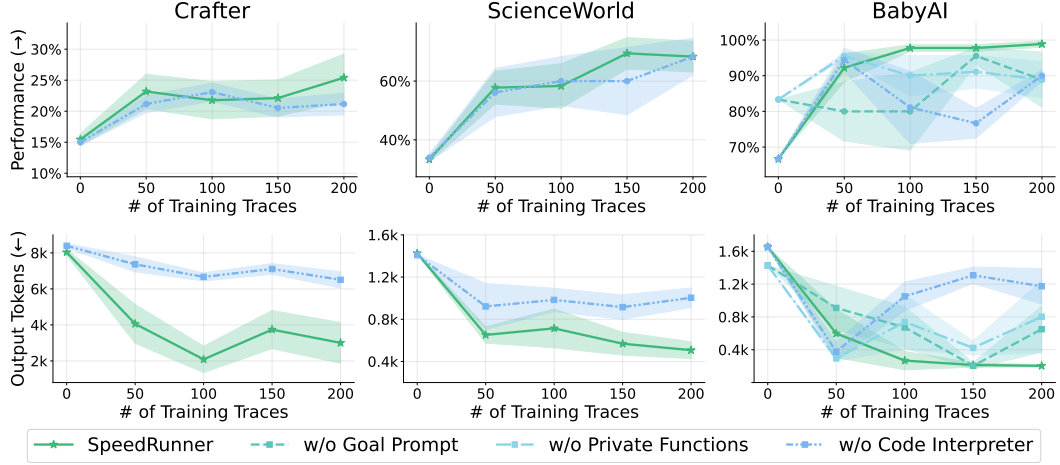


Figure 7: Extra SpeedRunner ablations on the Crafter and ScienceWorld environments. As in BabyAI, removing access to the code interpreter has only small effects on performance but contributes substantially to improvements in efficiency.

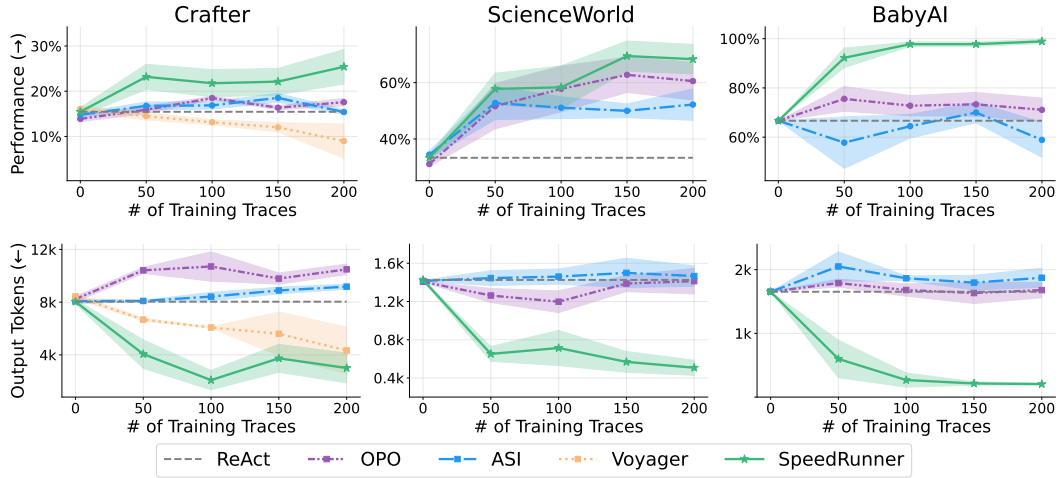


Figure 8: Main results using GPT-5.4-mini with tokens instead of cost for easy comparison across models.

evaluation is performed through the same rollout interface used by the other baselines, so that learned skills are assessed under the same acting protocol rather than through a bespoke execution regime.

G Additional Main Results

G.1 Code Interpreter Ablations

Our results in Figure 7 show that removing the code interpreter degrades final performance in two out of three environments and increases the amount of output tokens required substantially in all three. This demonstrates the importance of this component for SpeedRunner’s gains in performance and especially efficiency for programmatic skill induction.

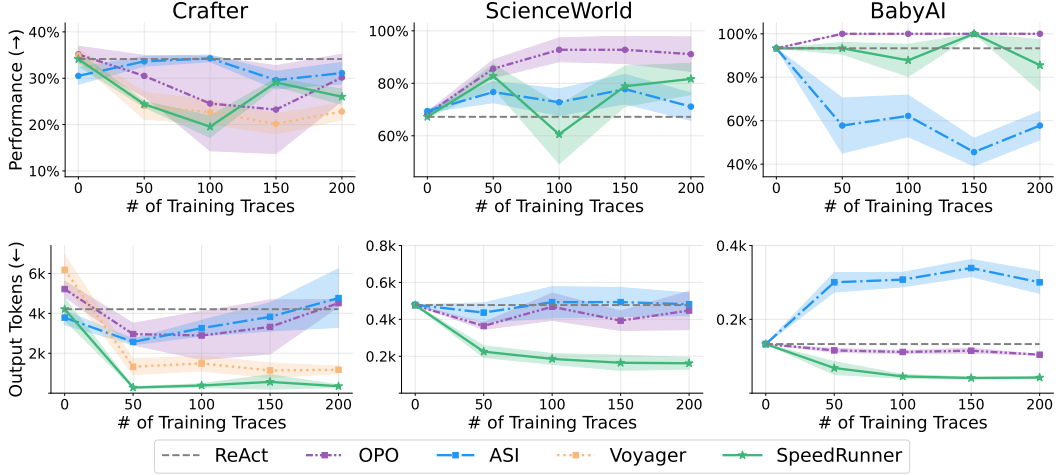


Figure 9: Gemini-3-Flash performance and output-token cost across Crafter, ScienceWorld, and BabyAI. As with GPT-5.4-mini, SpeedRunner reduces output tokens across the board, however, performance improvements are much less consistent than with GPT-5.4-mini.

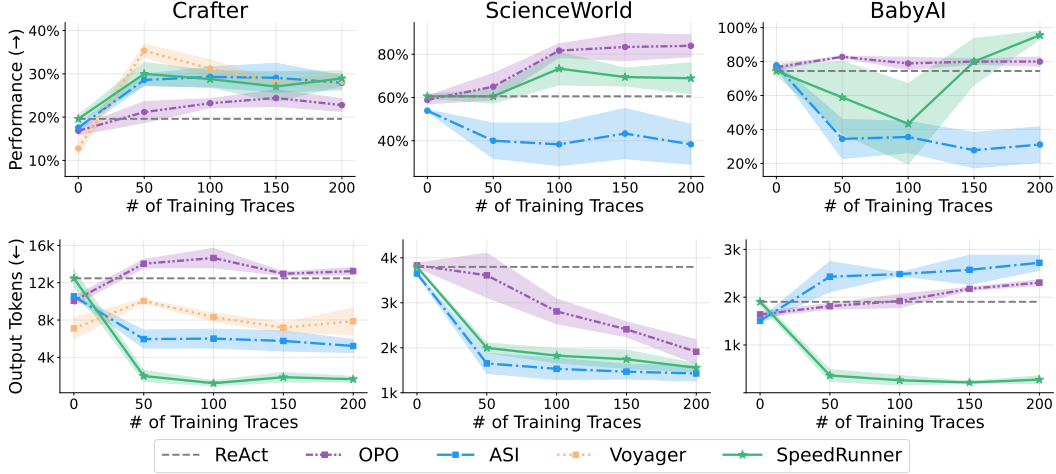


Figure 10: Qwen-3.5-27B performance and output-token cost across Crafter, ScienceWorld, and BabyAI. As with GPT-5.4-mini and Gemini-3-Flash, SpeedRunner generally matches or improves final task performance while reducing output tokens.

G.2 Cross-Model Results

In order to test whether SpeedRunner’s trends depend on the backbone LLM, we repeat the main comparison with Gemini-3-Flash and Qwen-3.5-27B. As shown in Figures 9 and 10, SpeedRunner retains the best output-token efficiency across the board and the best performance–efficiency tradeoff in most instances. More specifically, using the same two-sided pair t-test from our main results, we find that our method is significantly more efficient and better performing than our code-based baselines in all settings with every models except for Crafter, where they are much more competitive. Additionally, our method’s performance against OPO, our non-code baseline, reveals an interesting pattern. SpeedRunner achieves significantly worse final success rate than OPO with Qwen in ScienceWorld and Gemini in all benchmarks, despite achieving much lower token usage. We highlight this result because it shows that aggressive code delegation is not always performance-improving when the base actor is already strong. Not all performance can be transferred into code.

In an attempt to understand this behavior, we note that both Gemini-3-Flash and Qwen-3.5-27B start from higher performance levels than GPT-5.4-mini in all benchmarks where OPO beats SpeedRunner,

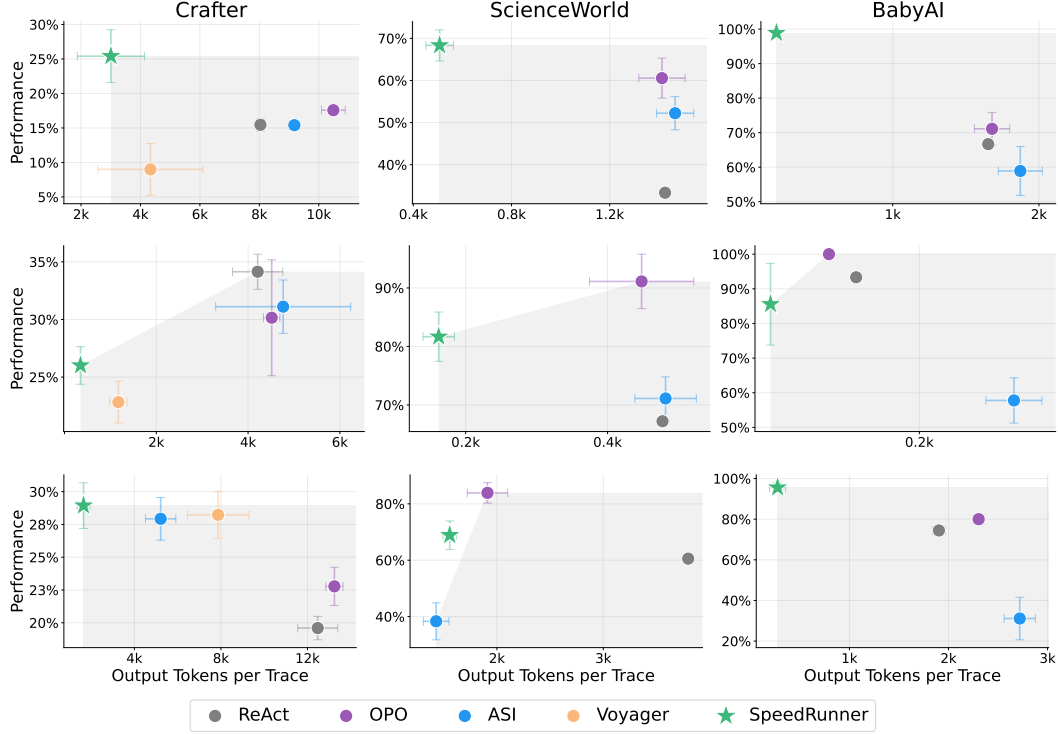


Figure 11: Performance vs output tokens per trace for all approaches and benchmarks for GPT-5.4-mini (top), Gemini-3-Flash (middle) and Qwen-3.5-27B (bottom) for easy comparison. This figure demonstrates that SpeedRunner dominates in most settings or remains at the frontier in all experiments we conducted.

indicating stronger raw model capacity in these environments. We believe this behavior is driven by compression aggressiveness. SpeedRunner’s iterative induction produces hierarchical libraries that delegate a large fraction of decisions to code; in Crafter, for example, Gemini-3-Flash reduces LLM calls by 94% over training. This yields very low inference cost, but it can also make the policy too rigid in stochastic environment. Code induced from earlier trajectories may encode procedures that are efficient on average but less able to adapt to later situations requiring reactive judgment. Thus, these results expose a real tradeoff: SpeedRunner can over-compress a capable actor, improving cost while sacrificing some performance.

Nevertheless, as illustrated in Figure 11, our approach succeeds in *consistent* learning and efficiency performance. Of the 9 settings, our approach dominates in 5 and is on the frontier in the remainder, a feat that is unmatched by any of the other baselines.

G.3 Offline Continual Learning with ASI

We further study the effect of experience replay by comparing our online adaptation and the original offline version of ASI. As described in Appendix E, the original algorithm verifies each new skill by re-executing the inducing task, whereas our online adaptation removes this step.

Figure 12 compare ASI, ASI (replay), and SpeedRunner on ScienceWorld task 3. Restoring the verification step improves ASI’s mean task performance, particularly early in training, because it filters out broken skills that lead to regression on a previous task. However, it does not reduce token cost beyond the adapted online version. After 200 training traces, SpeedRunner achieves higher mean task performance than both ASI variants while using substantially fewer output tokens per episode (approximately 65% fewer than both ASI variants). These results indicate that removing ASI’s replay-based verification step makes continual skill induction more challenging.

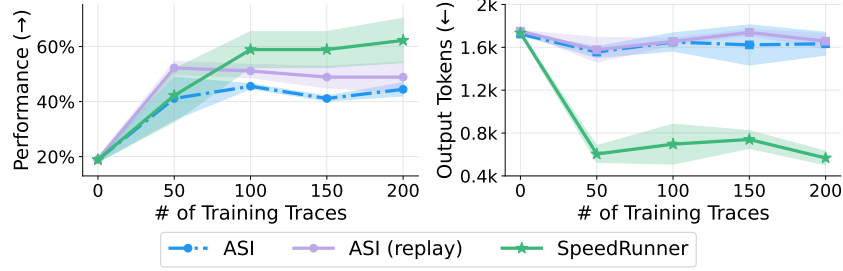


Figure 12: Effect of ASI’s replay-based skill verification on ScienceWorld task 3. Restoring replay-based verification improves ASI’s task performance, particularly early in training. SpeedRunner achieves the highest final performance while using substantially fewer output tokens than both ASI variants.

G.4 Why Performance Gains Vary by Benchmark

SpeedRunner’s main advantage comes from converting repeated behavior into compact executable routines, and the value of this conversion depends on the structure of the environment. ScienceWorld and BabyAI contain many reusable procedures that can be invoked repeatedly once discovered, so SpeedRunner improves performance while dramatically compressing output length. The effect is sharpest on BabyAI, where final per-eval-epoch output reaches roughly an *eighth* of the ReAct baseline — approaching an order-of-magnitude compression ratio that, combined with near-perfect final accuracy, indicates that the actor is almost entirely delegating decisions to a small set of induced abstractions rather than re-deriving primitive sequences. BabyAI’s high zero-shot baseline ($\sim 67\%$) limits how much headroom remains for raw performance gains, but this compression is itself the evidence that the code delegation pays off in reliability *and* efficiency.

Crafter is more challenging: the environment is longer-horizon, stochastic, and reactive, so a routine that is useful in one state can become inappropriate when immediate survival needs arise (e.g. a zombie attack). This shrinks the room for raw performance gains, but the compression advantage persists — SpeedRunner’s end-of-training output tokens are more than $3\times$ lower than OPO’s and ASI’s, even though all four methods begin from the same ReAct cost baseline and ASI and OPO actually grow rather than shrink. We analyze the structural reorganization that lets SpeedRunner retain this advantage as environmental randomness rises in §6.3.

H Full Quantitative Codebook Analysis

We provide the full structural analysis of the learned codebooks here. The main text summarizes the two most diagnostic call-graph measures, maximum depth and density; this appendix reports the complete set of statistics, including library size, average depth, and cyclomatic complexity.

Recall that we characterize the codebooks learned by each method using five structural measures, computed from the final codebook at the end of training. We first construct a directed call graph $G = (V, E)$ for each codebook, where each node $v \in V$ is an induced function and each edge $(u, v) \in E$ indicates that function u calls function v . Let $n = |V|$.

- **Library size** ($\#Fns$) is the number of induced functions, including both public and private functions.
- **Maximum depth** (*max d.*) is the length of the longest directed path in G , measuring the height of the deepest abstraction built by the inducer.
- **Average depth** (*avg d.*) is the mean depth across all induced functions in G .
- **Density** is $|E|/(n(n-1))$, excluding self-edges, and measures how often induced functions call other induced functions rather than relying only on the primitive action set.
- **Cyclomatic complexity** (*cyclo.*) is the standard McCabe complexity averaged across functions, capturing whether individual skills encode fixed action sequences or branch over multiple cases.

Table 2 shows that SpeedRunner learns compact but substantially more structured libraries than the baselines. Relative to ASI, SpeedRunner consistently increases call-graph density, maximum depth,

Benchmark	Method	#Fns	Max d.	Avg d.	Density	Cyclo.
Crafter	ASI	33.7 $\pm$ 7.2	1.7 $\pm$ 0.6	0.61 $\pm$ 0.06	0.030 $\pm$ 0.008	2.4 $\pm$ 1.1
	Voyager	1,077 $\pm$ 565	5.7 $\pm$ 2.1	0.44 $\pm$ 0.21	0.0005 $\pm$ 0.0003	30.2 $\pm$ 7.2
	SpeedRunner w/o CI	23.3 $\pm$ 11.5	6.3 $\pm$ 3.2	2.27 $\pm$ 1.33	0.138 $\pm$ 0.078	10.9 $\pm$ 5.1
	SpeedRunner (ours)	22.3 $\pm$ 4.0	8.7 $\pm$ 2.1	2.78 $\pm$ 0.96	0.149 $\pm$ 0.042	17.5 $\pm$ 3.2
ScienceWorld (task 4)	ASI	56.7 $\pm$ 42.1	1.3 $\pm$ 0.6	0.61 $\pm$ 0.26	0.015 $\pm$ 0.006	1.4 $\pm$ 0.6
	SpeedRunner w/o CI	25.3 $\pm$ 4.5	5.0 $\pm$ 1.0	1.38 $\pm$ 0.34	0.072 $\pm$ 0.007	11.7 $\pm$ 4.7
	SpeedRunner (ours)	30.0 $\pm$ 3.0	6.0 $\pm$ 1.0	1.99 $\pm$ 0.63	0.064 $\pm$ 0.015	9.1 $\pm$ 1.1
ScienceWorld (task 3)	ASI	73.0 $\pm$ 12.2	2.0 $\pm$ 0.0	0.84 $\pm$ 0.27	0.017 $\pm$ 0.002	1.0 $\pm$ 0.1
	SpeedRunner w/o CI	31.3 $\pm$ 6.7	5.3 $\pm$ 1.5	1.63 $\pm$ 0.39	0.074 $\pm$ 0.018	9.1 $\pm$ 2.5
	SpeedRunner (ours)	30.7 $\pm$ 1.5	5.7 $\pm$ 0.6	1.84 $\pm$ 0.12	0.069 $\pm$ 0.020	8.5 $\pm$ 2.1
BabyAI	ASI	25.3 $\pm$ 6.7	1.0 $\pm$ 0.0	0.41 $\pm$ 0.12	0.018 $\pm$ 0.005	2.0 $\pm$ 1.6
	SpeedRunner w/o CI	9.0 $\pm$ 4.0	3.3 $\pm$ 1.5	1.31 $\pm$ 0.70	0.178 $\pm$ 0.019	8.1 $\pm$ 1.1
	SpeedRunner (ours)	5.7 $\pm$ 0.6	2.3 $\pm$ 0.6	0.73 $\pm$ 0.40	0.206 $\pm$ 0.042	10.9 $\pm$ 1.7

Table 2: Structural measures of each method’s final skill library across four benchmark settings, using GPT-5.4-mini with mean $\pm$ sd over three seeds. Shaded rows mark the full SpeedRunner system. Bold marks the most compact library for #Fns and the highest value for the remaining structural measures within each benchmark.

and average depth, indicating that its skills are not merely shallow wrappers around primitive actions but reusable routines with hierarchical dependencies. Relative to Voyager on Crafter, SpeedRunner achieves much higher density with nearly two orders of magnitude fewer functions, suggesting that it factors repeated behavior into shared helpers instead of accumulating scenario-specific code.

Although Voyager has higher cyclomatic complexity on Crafter, this complexity is concentrated inside individual functions rather than distributed through reusable library structure. Its density is only 0.0005, compared with 0.149 for SpeedRunner, meaning that its functions rarely call one another. Thus, Voyager’s library is large and internally branchy, but weakly compositional: each function tends to encode a complete scenario rather than decomposing behavior into smaller reusable components.

The code-interpreter ablation preserves much of SpeedRunner’s organization, but the full method is generally deeper, denser, or more decision-rich. On Crafter and both ScienceWorld settings, SpeedRunner has higher maximum and average depth than SpeedRunner w/o CI. On BabyAI, where the tasks are shorter and the learned libraries are naturally small, SpeedRunner instead has the highest density and cyclomatic complexity. Overall, SpeedRunner’s advantage is not larger codebooks, but better organized ones: compact libraries whose skills are more reusable, modular, and compositional.

I Per-Environment Qualitative Analysis

This appendix expands the three patterns identified in the main paper with concrete examples from each environment. Each subsection contrasts the key abstraction discovered by ASI, the code-interpreter-free ablation (SpeedRunner w/o CI), and SpeedRunner.

I.1 BabyAI: Compositional Mission Parsing

The PickupSeqGoTo task requires parsing a natural-language mission (e.g., “go to the red box after you pick up the blue ball”) and executing it in a partially-observable grid world. Some phrasings reverse grammatical order relative to execution order, which is the decisive challenge.

A naïve left-to-right parser inverts execution order on reversed-phrasing missions. SpeedRunner w/o CI has no parser and leaves disambiguation to the LLM, which occasionally misorders goals (its 6.7% error). SpeedRunner’s coding agent iterated over historical mission strings inside `execute_code`, observed the inversion, and added reordering entries to a priority-ordered pattern list (matching “go to X after you pick up Y” and “pick up X after you go to Y” before the fallback). The gap between SpeedRunner w/o CI and SpeedRunner is entirely attributable to this one structural decision.

Table 3: Key abstractions per method in the BabyAI environment. The task hinges on parsing missions where grammatical order can reverse execution order. ASI templates motion sequences from traces and never parses the mission; SpeedRunner w/o CI patches navigation failures reactively but still defers mission parsing to the LLM; only SpeedRunner derives an explicit priority-ordered parser by executing candidate patterns against historical mission strings.

Method	Key abstraction
ASI	Fixed motion sequences (repeat_left_step_and_look, move_left_then_pick_up) extracted verbatim from successful traces. No function reads observations or branches on state.
SpeedRunner w/o CI	_approach_object encodes three failure modes (target hidden, blocked, laterally offset), each guard added one sleep phase after the failure that revealed it. No mission parser.
SpeedRunner	_parse_mission_goals: priority-ordered pattern list where reversed-order phrasings are matched before the left-to-right fallback. Discovered by running candidate parsers against historical mission strings via execute_code.

I.2 Crafter

Table 4: Crafter: key abstractions per method, showing how each handles the tech-tree progression and resource-lookup failures characteristic of the environment. ASI crafts unconditionally with no inventory checks; Voyager appends a new function per episode and never consolidates, yielding 483 near-duplicate variants; SpeedRunner w/o CI builds reactive guards one failure at a time; SpeedRunner encodes the full progression hierarchy in a single function and diagnoses systematic lookup failures via trace queries to introduce _bounded_exploration. The Voyager row illustrates that observation-reading alone is insufficient—consolidation is a necessary co-condition.

Method	Key abstraction
ASI	prepare_basic_wood_tools crafts unconditionally regardless of inventory; observation calls at function ends are trace artefacts.
SpeedRunner w/o CI	bootstrap_stone_pickaxe gates stone behind a wood-pickaxe check; hostile-check before rest in stabilize_visible_survival added reactively after deaths. Guards accumulate one failure at a time.
SpeedRunner	_choose_progress_target encodes the full tech-tree hierarchy in one function, enumerated via execute_code before any episode. _bounded_exploration added after the inducer diagnosed a high rate of failed resource lookups across batches.
Voyager	New function appended per episode; no editing. 212 near-identical collect_one_wood_log_* variants. Same 14-line observation-parsing block copy-pasted into every function.

SpeedRunner’s _choose_progress_target encodes the full tech tree in one place: higher-tier ores are gated behind the corresponding pickaxe tier, and stone is de-prioritized once the stockpile is sufficient. Neither ASI nor SpeedRunner w/o CI has an analogue—ASI crafts unconditionally, and SpeedRunner w/o CI’s guards are reactive and incomplete.

The second decisive abstraction, _bounded_exploration, illustrates the inducer’s diagnostic loop. After observing repeated not_found returns from travel_to_visible_resource, the inducer queried the last batch of traces and printed every top-level not_found alongside its target (wood, tree, cow). It then walked the trace tree to count call frequencies per batch and observed that travel_to_visible_resource was being invoked 116–250 times per batch in later rounds—i.e., the agent was repeatedly trying and failing to find resources in view. Diagnosing this as a visibility problem rather than a navigation bug, the inducer added _bounded_exploration, which sweeps the local neighborhood around the agent for the desired resource before falling back to a not_found.

Voyager is the counterfactual to consolidation: 483 functions, no shared progression model, boilerplate duplicated hundreds of times, and the lowest score of any method (14.7%, below ASI’s 20.5%). Reading observations is not sufficient; consolidation is a necessary co-condition.

Table 5: ScienceWorld T3 (conductivity): key abstractions per method. The task requires wiring an unknown substance into a battery-and-bulb circuit and placing it in the correct box based on whether the bulb lights. The decisive ASI–ablation gap is programmatic result reading: ASI places the substance after a fixed wait regardless of bulb state, while SpeedRunner w/o CI gates placement on a regex over the bulb observation. SpeedRunner’s additional contribution is consolidation—hoisting per-function ambiguity handling into a single wrapper, cutting library size by 22%.

Method	Key abstraction
ASI	Polarity functions <code>setup_conductivity_test</code> and <code>finish_conductivity_check</code> added sequentially after wiring failures. Neither clears prior circuit state; placement is unconditional after a fixed <code>wait()</code> .
SpeedRunner w/o CI	<code>_test_conductivity_with_circuit</code> reads the bulb via <code>re.search("which is on")</code> ; <code>_reset_circuit</code> fully disconnects between attempts. Four polarity patterns added reactively.
SpeedRunner	Same programmatic result reading as SpeedRunner w/o CI, plus a single <code>_call_one_arg_action_with_disambiguation</code> wrapper that routes every primitive call through one ambiguity handler. Room search uses DFS up to 24 steps—discovered via <code>execute_code</code> for cases where the target is not in the stated room.

I.3 ScienceWorld: Programmatic Verification

ScienceWorld T3 (conductivity) requires wiring an unknown substance into a battery-and-bulb circuit and placing it in the correct box. T4 (find category) requires locating an object of a given category across rooms.

The decisive gap between ASI and SpeedRunner w/o CI is programmatic result reading: ASI moves to the target box after a fixed wait regardless of bulb state; SpeedRunner w/o CI gates placement on `re.search("which is on")` of the bulb observation. SpeedRunner’s additional contribution is consolidation—in SpeedRunner w/o CI, ambiguity handling was added ad-hoc inside each function as the error appeared; SpeedRunner hoisted it into a single wrapper, cutting library size 22% (27 → 21) while applying the fix uniformly.

Table 6: ScienceWorld T4 (find category): key abstractions per method, the one setting where SpeedRunner w/o CI outperforms SpeedRunner. Both methods program category classification—a clear advance over ASI’s full delegation to the LLM—but with different strategies: SpeedRunner w/o CI maintains targeted exclusion lists for observed false positives (“painting,” “egg”), while SpeedRunner derives general normalization rules via `execute_code`. The result suggests pre-commit testing helps most when the failure-mode space is combinatorially large; for finite enumerable false-positive sets, reactive per-failure refinement is sufficient.

Method	Key abstraction
ASI	<code>focus_roundtrip_pick_move</code> : round-trip navigation template from traces. Category classification fully delegated to the LLM.
SpeedRunner w/o CI	<code>_extract_animal_name</code> with priority-ordered patterns and artwork/egg exclusions, derived from false-positive failures (“painting of a bee”). <code>_try_pickup</code> verifies retrievability before returning a match.
SpeedRunner	<code>_canonicalize_object_name</code> strips stage labels (“in the dead stage”) via <code>execute_code</code> -tested rules. <code>_probe_bee_hive_for_target</code> added after observing missed hive contents.

T4 is the one setting where SpeedRunner w/o CI outperforms SpeedRunner. Both implement programmatic category classification—a clear advance over ASI’s delegation—but with different strategies: SpeedRunner w/o CI uses targeted exclusion lists (skip “painting,” “egg”), while SpeedRunner uses general normalization rules. That SpeedRunner w/o CI still wins suggests pre-commit testing helps most when the failure-mode space is *combinatorially large*—as in BabyAI’s mission ordering or Crafter’s tech tree—rather than a finite enumerable set of false-positive patterns, where reactive per-failure refinement is sufficient.

Method	Cond.	#Fns	Max d.	Avg d.	Density	Cyclo.	%Combat
Voyager	0x	1,368	5	0.40	0.0002	33.4	0.7%
Voyager	1x	1,077	5.7	0.44	0.0005	30.2	2.9%
Voyager	2x	590	9	0.52	0.0007	31.6	11.5%
SpeedRunner	0x	25	6	2.40	0.132	11.2	24.0%
SpeedRunner	1x	22	8.7	2.78	0.149	17.5	31.5%
SpeedRunner	2x	32	10	2.62	0.094	14.8	53.1%

Table 7: Final-library structure across the three randomness conditions (GPT-5.4-mini). Columns are the structural measures from §H; %Combat is the fraction of induced functions whose body references a creature/combat keyword.

Visible N	0 (none)	50	100	200	400	800	909 (full)
Mean Progression	0.173	0.124	0.129	0.130	0.174	0.127	0.153

Table 8: Post-hoc truncation of Voyager’s final 1×-zombie library (909 induced skills). The rollout agent is shown only the most recently induced N skills; earlier skills remain callable when invoked from inside a visible one. Mean Progression is the fraction of the 22 Crafter achievements unlocked per episode, averaged over the 30-episode held-out test set. $N = 0$ and $N = 909$ are the no-codebook and full-library reference points taken from the training-time periodic evaluation of the same run.

J External Randomness: Additional Analysis

Table 7 reports the full structural measures (definition in §H) that the figure abstracts over.

The mechanism behind SpeedRunner’s %Combat rise is concretely visible in the 2x library: of its ~ 17 combat-aware functions, eleven are *existing* harvest and movement routines (e.g., `harvest_visible`, `clear_front_blocker`, `opening_turn`) that have been edited to accept an `include_combat` or `allow_combat` keyword argument — one kwarg flip and every caller gets the defensive variant. Voyager, unable to edit, instead spawns a fresh bespoke function for each new encounter (`collect_wood_log_avoid_front_zombie`, `collect_one_log_avoiding_east_zombie`, `step_adjacent_to_skeleton`, ...): the same combat logic that SpeedRunner amortizes once across its call graph is paid in duplicated branches inside every Voyager skill, which is why Voyager’s per-skill cyclomatic remains the highest of any method even as its density stays near zero.

K Post-hoc Library Truncation on Voyager

A natural reading of Voyager’s regression under standard (1×) and doubled (2×) zombie pressure (Table 7) is that its append-only library simply grows past the point where the rollout agent can usefully attend to it: by the end of training, the GPT-5.4-mini seeds carry libraries of {615, 909, 1707} public skills, and one of them already overflows the model’s context window outright. If volume alone were the issue, then keeping only the most recently induced skills should recover most of the gap to the no-codebook baseline. We therefore ask the following question: can a simple last- N truncation of Voyager’s final library rescue its 1×-zombie performance?

Setup. We take the final codebook checkpoint of one 1×-zombie Voyager seed and re-evaluate it on the same 30-episode Crafter test set used in the main results (§5), holding the rollout agent and seeds fixed. For each $N \in \{50, 100, 200, 400, 800\}$, we present the rollout agent with only the *last* N skills—those induced most recently—as callable tools and as entries in the instruction manual; the earlier $909 - N$ skills are hidden from the prompt but remain in the execution namespace, so any visible skill whose body invokes an earlier one still runs correctly. This isolates the variable of interest: the size of the *agent-visible* library, with the underlying behavior of each visible skill held fixed. We bracket the sweep with two reference points reused from the training-time periodic evaluation of the same run: $N = 0$ corresponds to the no-codebook ReAct baseline (the actor runs with primitives only), and $N = 909$ corresponds to the full final library.

Truncation does not structurally remedy Voyager. Three observations from Table 8 point against volume being the binding constraint. First, the truncation curve is *non-monotonic* in N : there is no clear rising or falling trend as the visible library shrinks. Second, *no truncation meaningfully exceeds the no-codebook baseline*: the closest setting, $N = 400$, matches it within noise (a gap of one achievement unlocked on one episode), and every other setting falls below. No choice of N —including the smallest visible surface we test—turns Voyager’s library into a net asset relative to running with primitives only. Third, the full library also underperforms the no-codebook baseline on this seed, consistent with the regression pattern reported in Table 7.

Taken together, these results indicate that Voyager’s regression under standard zombie density is not a pure context-bloat phenomenon; the *composition* of the induced library matters more than the headcount of skills exposed at rollout time.

L Use of AI Assistants

We used ChatGPT and Claude as assistants for writing (grammar correction, clarity edits, and suggestions on presentation) and coding (implementation and debugging of experimental infrastructure). All scientific ideas, experimental designs, analyses, claims, and final text were reviewed and approved by the authors.